

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря
Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

**Лабораторна робота №1
з дисципліни
«Об'єктно-орієнтоване програмування»
на тему**

Знайомство із середовищем розробки та складання модульних проєктів програм

Виконала
студентка групи ІМ-о51
Глазкова Ілона Андріївна
Номер в списку - 1

Перевірів
Порєв В.М.

Завдання:

1. Створити у обраному середовищі розробки проєкт з ім'ям Lab1.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпільовати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. За потреби налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.

Завдання згідно варіанту:

1. Вікно діалогу з повзунок горизонтального скролінгу (Horizontal scroll Bar) та дві кнопки: [Так] і [Відміна]. Рухаючи повзунок скролінгу користувач вводить число у діапазоні від 1 до 100. Після натискування кнопки [Так] вибране число буде відображатися у головному вікні.

2. Два вікна діалогу. Спочатку з'являється перше, яке має дві кнопки: [Далі >] і [Відміна]. Якщо натиснути кнопку [Далі >], то воно закриється і з'явиться друге діалогове вікно, яке має кнопки: [< Назад], [Так] і [Відміна]. Якщо натиснути кнопку [<Назад], вікно закриється і перехід до першого вікна.

Код:

```
...  
  
#include "framework.h"  
#include "Lab1.h"  
#include <stdio.h>  
  
#define MAX_LOADSTRING 100  
  
// Global Variables:  
HINSTANCE hInst;                // current instance  
WCHAR szTitle[MAX_LOADSTRING]; // The title bar text  
WCHAR szWindowClass[MAX_LOADSTRING]; // the main window class name  
  
// Forward declarations of functions included in this code module:  
ATOM MyRegisterClass(HINSTANCE hInstance);  
BOOL InitInstance(HINSTANCE, int);  
LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);  
INT_PTR CALLBACK About(HWND, UINT, WPARAM, LPARAM);  
INT_PTR CALLBACK Robota1(HWND, UINT, WPARAM, LPARAM);  
INT_PTR CALLBACK Robota2_Window1(HWND, UINT, WPARAM, LPARAM);  
INT_PTR CALLBACK Robota2_Window2(HWND, UINT, WPARAM, LPARAM);  
  
HWND hStatic = 0;  
  
int APIENTRY wWinMain(_In_ HINSTANCE hInstance,  
                     _In_opt_ HINSTANCE hPrevInstance,  
                     _In_ LPWSTR lpCmdLine,
```

```

        _In_ int    nCmdShow)
{
    UNREFERENCED_PARAMETER(hPrevInstance);
    UNREFERENCED_PARAMETER(lpCmdLine);

    // TODO: Place code here.

    // Initialize global strings
    LoadStringW(hInstance, IDS_APP_TITLE, szTitle, MAX_LOADSTRING);
    LoadStringW(hInstance, IDC_LAB1, szWindowClass, MAX_LOADSTRING);
    MyRegisterClass(hInstance);

    // Perform application initialization:
    if (!InitInstance (hInstance, nCmdShow))
    {
        return FALSE;
    }

    HACCEL hAccelTable = LoadAccelerators(hInstance,
MAKEINTRESOURCE(IDC_LAB1));

    MSG msg;

    // Main message loop:
    while (GetMessage(&msg, nullptr, 0, 0))
    {
        if (!TranslateAccelerator(msg.hwnd, hAccelTable, &msg))
        {
            TranslateMessage(&msg);
            DispatchMessage(&msg);
        }
    }

    return (int) msg.wParam;
}

```

```

//
// FUNCTION: MyRegisterClass()
//
// PURPOSE: Registers the window class.
//
ATOM MyRegisterClass(HINSTANCE hInstance)
{
    WNDCLASSEXW wcex;

    wcex.cbSize = sizeof(WNDCLASSEX);

```

```

wcex.style      = CS_HREDRAW | CS_VREDRAW;
wcex.lpfnWndProc = WndProc;
wcex.cbClsExtra  = 0;
wcex.cbWndExtra  = 0;
wcex.hInstance  = hInstance;
wcex.hIcon      = LoadIcon(hInstance, MAKEINTRESOURCE(IDI_LAB1));
wcex.hCursor    = LoadCursor(nullptr, IDC_ARROW);
wcex.hbrBackground = (HBRUSH)(COLOR_WINDOW+1);
wcex.lpszMenuName = MAKEINTRESOURCEW(IDC_LAB1);
wcex.lpszClassName = szWindowClass;
wcex.hIconSm    = LoadIcon(wcex.hInstance, MAKEINTRESOURCE(IDI_SMALL));

return RegisterClassExW(&wcex);
}

//
// FUNCTION: InitInstance(HINSTANCE, int)
//
// PURPOSE: Saves instance handle and creates main window
//
// COMMENTS:
//
//   In this function, we save the instance handle in a global variable and
//   create and display the main program window.
//
BOOL InitInstance(HINSTANCE hInstance, int nCmdShow)
{
    hInst = hInstance; // Store instance handle in our global variable

    HWND hWnd = CreateWindowW(szWindowClass, szTitle, WS_OVERLAPPEDWINDOW,
        CW_USEDEFAULT, 0, CW_USEDEFAULT, 0, nullptr, nullptr, hInstance, nullptr);

    if (!hWnd)
    {
        return FALSE;
    }

    hStatic = CreateWindowW(
        L"STATIC",
        L"",
        WS_CHILD | WS_VISIBLE | SS_LEFT,
        10, 10, 200, 30,
        hWnd,
        (HMENU)1,
        hInstance,
        nullptr
    );

```

```

ShowWindow(hWnd, nCmdShow);
UpdateWindow(hWnd);

return TRUE;
}

```

```

void Work2(HWND hwnd)
{

}

```

```

//
// FUNCTION: WndProc(HWND, UINT, WPARAM, LPARAM)
//
// PURPOSE: Processes messages for the main window.
//
// WM_COMMAND - process the application menu
// WM_PAINT - Paint the main window
// WM_DESTROY - post a quit message and return
//
//
LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam,
LPARAM lParam)
{
    switch (message)
    {
        case WM_MY_CUSTOM_MESSAGE_FROM_ROBOTA1:
        {
            int receivedValue = (int)wParam;
            char textBuffer[32];
            sprintf_s(textBuffer, 32, "Value: %d", receivedValue);
            SetWindowTextA(hStatic, textBuffer);
            InvalidateRect(hStatic, NULL, TRUE);
            UpdateWindow(hStatic);
            break;
        }
        case WM_COMMAND:
        {
            int wmlId = LOWORD(wParam);
            // Parse the menu selections:
            switch (wmlId)
            {
                case ID_ACTIONS_ROBOTA2:
                    ShowWindow(hWnd, SW_HIDE);

```

```

        DialogBox(hInst, MAKEINTRESOURCE(IDD_Work2_DIALOG1), hWnd,
Robota2_Window1);
        break;
    case ID_ACTIONS_ROBOTA1:
        ShowWindow(hWnd, SW_HIDE);
        DialogBox(hInst, MAKEINTRESOURCE(IDD_Work1_DIALOG), hWnd, Robota1);
        break;
    case IDM_ABOUT:
        DialogBox(hInst, MAKEINTRESOURCE(IDD_ABOUTBOX), hWnd, About);
        break;
    case IDM_EXIT:
        DestroyWindow(hWnd);
        break;
    default:
        return DefWindowProc(hWnd, message, wParam, lParam);
    }
}
break;
case WM_PAINT:
{
    PAINTSTRUCT ps;
    BeginPaint(hWnd, &ps);
    // TODO: Add any drawing code here...
    EndPaint(hWnd, &ps);
}
break;
case WM_DESTROY:
    PostQuitMessage(0);
    break;
default:
    return DefWindowProc(hWnd, message, wParam, lParam);
}
return 0;
}

```

// Message handler for about box.

INT_PTR CALLBACK About(HWND hDlg, UINT message, WPARAM wParam, LPARAM lParam)

```

{
    UNREFERENCED_PARAMETER(lParam);
    switch (message)
    {
    case WM_INITDIALOG:
        return (INT_PTR)TRUE;

    case WM_COMMAND:
        if (LOWORD(wParam) == IDOK || LOWORD(wParam) == IDCANCEL)
        {

```

```
        EndDialog(hDlg, LOWORD(wParam));  
        return (INT_PTR)TRUE;  
    }  
    break;  
}  
return (INT_PTR)FALSE;  
}
```